

IAM-96 LAB: Python IAM Provisioning

— Groups, Policies, Boundaries, Bindings

Series: IAM — IAM Administration | **Reference:** 96 — Python IAM Provisioning LAB | **Created:** July 2026 | **Last Updated:** 07/16/2026

Overview

Hands-on lab: onboard teams into a persona + team IAM model with **Python** using the [Python-IAM-Group-Policy-MZ-Boundry](#) tool. One command creates the group, the shared parameterized policy, the per-team boundary, and the binding — idempotently. Every expected output below comes from a real run, verified live against the Account Management API on 07/16/2026.

This is one of **three provisioning flavors**:

Flavor	Notebook	Best for
curl / shell	IAM-12	One-off scripts, CI glue
Terraform	IAM-95	Declarative IaC, drift detection, plan/apply review
Python (this lab)	IAM-96	Programmatic onboarding, portals, ad-hoc admin

What one `--team` command builds (the BPN pattern from IAM-05 / IAM-10):

```
1. GROUP      "&lt;Team&gt; Team"                (created or found)
2. POLICY     tpl-team-data-access            (created ONCE, reused by
every team)
              ALLOW storage:logs:read WHERE storage:dt.security_context IN
              ("${bindParam:team}"); ...
3. BOUNDARY   bnd-team-&lt;security-context&gt;    (one per team,
defense-in-depth)
4. BINDING    group → policy with parameters={"team": "&lt;security-
context&gt;"} + boundary
```

Time: 20–30 minutes | **Difficulty:** Beginner-friendly | **Cost:** No consumption impact (IAM objects only)

Table of Contents

1. [Setup](#)
 2. [Preview a Team Onboarding \(Dry Run\)](#)
 3. [Execute](#)
 4. [Onboard a Second Team \(Watch the Policy Get Reused\)](#)
 5. [Prove Idempotency](#)
 6. [Validate](#)
 7. [Clean Up](#)
 8. [Troubleshooting](#)
 9. [Validation Checklist](#)
 10. [References](#)
-

Prerequisites

Requirement	Details
Python	3.10+ (<code>python3 --version</code>)
Project	<code>git clone git@github.com:timstewart-dynatrace/Python-IAM-Group-Policy-MZ-Boundry.git</code>

Requirement	Details
Account UUID	Account Management → Account settings
OAuth client	Account Management → Identity & access management → OAuth clients
OAuth scopes	<code>account-idm-read account-idm-write iam-policies-management</code> <code>iam:groups:read iam:policies:read iam:policies:write</code> <code>iam:bindings:read iam:bindings:write iam:boundaries:read</code> <code>iam:boundaries:write account-env-read</code>
Prior reading	IAM-04 (policy authoring), IAM-05 (boundary design), IAM-10 (templated policies)

⚠ Safety rails (do not skip):

1. Run only against a **sanctioned test tenant** — confirm the environment ID before executing.
2. Use a `PYTEST-` name prefix for lab objects so they are unmistakable.
3. Always pass `--level-type environment --level-id <env-id>` .
Groups/policies/boundaries are **account-level** objects; the binding grants access, and environment-scoping it confines the lab to one environment.
4. Every command below runs as a **dry run by default** — nothing changes until you add `--execute` .

Step 1 — Setup

```
git clone git@github.com:timstewart-dynatrace/Python-IAM-Group-Policy-MZ-Boundary.git
cd Python-IAM-Group-Policy-MZ-Boundary
pip install -r requirements.txt
cp .env.example .env # then edit with your account UUID + OAuth client
```

Sanity-check credentials with a read-only call:

```
python setup_group.py --list-boundaries
```

Expected output (shape):

Policy Boundaries:

```
<existing-boundary-name>
  UUID: 469072db-e3fd-4df3-975a-60412fc9b388
  Query: environment:management-zone IN ("...");...
```

Total: 5 boundaries

✅ **Checkpoint:** a count prints without a 401 — your OAuth client works.

Step 2 — Preview a Team Onboarding (Dry Run)

```
python setup_group.py --team "PYTEST-Gamma" --security-context pytest-gamma \
  --management-zone "PYTEST-Gamma" \
  --policy-name "PYTEST-tpl-team-data-access" \
  --level-type environment --level-id abc12345
```

Expected output:

```
=====
DRY RUN - No changes will be made
=====

Team:                PYTEST-Gamma
Security context:    pytest-gamma
Management zone:     PYTEST-Gamma
Binding level:       environment (abc12345)

-----
Status: DRY_RUN
Message: Would onboard team 'PYTEST-Gamma': group 'PYTEST-Gamma Team' bound
to
'PYTEST-tpl-team-data-access' with team=pytest-gamma and boundary 'bnd-team-
pytest-gamma'

Steps completed: group_dry_run, policy_dry_run, boundary_dry_run,
binding_dry_run
```

Step 3 — Execute

Add `--execute` to the same command. **Expected output:**

```
Status: SUCCESS
Message: Team 'PYTEST-Gamma' onboarded: group 'PYTEST-Gamma Team' bound to
'PYTEST-tpl-team-data-access' with team=pytest-gamma and boundary 'bnd-team-
pytest-gamma'

Steps completed: group_created, policy_created, boundary_created,
binding_bound
Group: CREATED
  UUID: 864aad76-a7de-41a4-9f23-9894aa7241eb
Policy: CREATED
  UUID: 86c52dfc-deb9-4b88-84fe-3cb3cb043b3b
Boundary: CREATED
  UUID: 52831d58-7e3d-470a-99ec-285de6e20140
Binding: BOUND
```

Step 4 — Onboard a Second Team (Watch the Policy Get Reused)

```
python setup_group.py --team "PYTEST-Delta" --security-context pytest-delta \
  --policy-name "PYTEST-tpl-team-data-access" \
  --level-type environment --level-id abc12345 --execute
```

Expected output — note `policy_exists` :

```
Steps completed: group_created, policy_exists, boundary_created,
binding_bound
```

✅ **Checkpoint:** the second team did **not** create a second policy. One template, many bindings — that is the IAM-10 pattern working.

Step 5 — Prove Idempotency

Re-run the exact Step 3 command. **Expected output:**

```
Status: SUCCESS
Steps completed: group_exists, policy_exists, boundary_exists,
binding_unchanged
Binding: UNCHANGED
```

Nothing errored, nothing duplicated — the tool converges. (Under the hood, the API answers 400 "already exists" when the binding matches the desired state; the tool treats that as convergence. See IAM-12 §7.)

Step 6 — Validate

In the UI — Account Management → Identity & access management: Groups (search PYTEST-), Policies (open PYTEST-tpl-team-data-access → see literal `${bindParam:team}` ; bindings tab shows resolved values), Boundaries.

Via Python:

```
import requests
from dynatrace_group_setup import DynatraceGroupSetup, load_env

load_env()
s = DynatraceGroupSetup()
r =
requests.get("https://api.dynatrace.com/iam/v1/repo/environment/abc12345/bind
ings",
            headers=s.headers)
group = s.get_group_by_name("PYTEST-Gamma Team")
for b in r.json()["policyBindings"]:
    if group["uuid"] in b.get("groups", []):
        print(b["parameters"], b["boundaries"])
```

Expected output:

```
{'team': 'pytest-gamma'} ['52831d58-7e3d-470a-99ec-285de6e20140']
```

✓ **Checkpoint:** `parameters.team` equals the security context; exactly one boundary attached.

Step 7 — Clean Up

Deletion order matters: **group first** (dropping its bindings), then policy, then boundary.

```
from dynatrace_group_setup import DynatraceGroupSetup, load_env
load_env()
s = DynatraceGroupSetup()

for team, ctx in [("PYTEST-Gamma", "pytest-gamma"), ("PYTEST-Delta", "pytest-
delta")]:
    g = s.get_group_by_name(f"{team} Team")
    if g: print("group:", s.delete_group(g["uuid"])["status"])
    b = s.get_boundary_by_name(f"bnd-team-{ctx}")
    if b: print("boundary:", s.delete_boundary(b["uuid"])["status"])

p = s.get_policy_by_name("PYTEST-tpl-team-data-access", level_type="account")
if p: print("policy:", s.delete_policy(p["uuid"])["status"])
```

Expected output:

```
group: DELETED
boundary: DELETED
group: DELETED
boundary: DELETED
policy: DELETED
```

Verify with `python setup_group.py --list-groups | grep PYTEST` → no results.

Troubleshooting

Symptom	Cause	Fix
Error: Missing environment variables	<code>.env</code> absent or incomplete	Copy <code>.env.example</code> → <code>.env</code> , fill all four values
401/ Invalid OAuth2 credentials	Wrong client secret or scopes	Recreate the OAuth client with the full scope list
400 Invalid permission name when creating a custom policy	Nonexistent permission or wildcard	See verified rules in IAM-04 and the error table in IAM-12 §7

Symptom	Cause	Fix
Binding: <code>ERROR ...</code> already exists on old tool versions	Pre-07/2026 version without convergence handling	<code>git pull</code> — re-runs now report <code>UNCHANGED</code>
Policy deletion 400	Policy still bound to a group	Delete the group (drops bindings) before the policy

Validation Checklist

- [] Read-only `--list-boundaries` worked before any writes
- [] Dry run showed all four steps as `*_dry_run`
- [] First team: `policy_created` · second team: `policy_exists` (shared template)
- [] Re-run reported `binding_unchanged` — no duplicates, no errors
- [] API check showed `parameters.team` + exactly 1 boundary
- [] Cleanup removed every `PYTEST-` object

References

- **IAM-04** Policy Authoring (verified statement syntax) · **IAM-05** Boundary Design · **IAM-10** Templated Policy Assignments · **IAM-12** API Provisioning (curl flavor) · **IAM-95** Terraform lab (same pattern, IaC flavor)
- **MZ2POL series** — migrating off Management Zones onto this pattern
- Repo: [Python-IAM-Group-Policy-MZ-Boundary](#) · Reference utility: [Python-IAM-Utility-3.0 \(dtiam\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.